

# Proposal: CBOM Support for Dependency-Track

## Cryptographic Bill of Materials: Post-Quantum Readiness & Export Control

### 1. Why CBOM Matters for Dependency-Track

Dependency-Track has established itself as the leading open-source platform for Software Bill of Materials (SBOM) management. We believe extending it with **Cryptographic Bill of Materials (CBOM)** support represents a natural and high-value evolution that serves two urgent, converging regulatory drivers:

#### 1.1 Post-Quantum Cryptography Migration

NIST finalized the first post-quantum cryptographic standards in 2024 (FIPS 203, 204, 205). Organizations worldwide now face a multi-year migration away from RSA, ECDSA, and other quantum-vulnerable algorithms. To plan this migration, they must first **inventory every cryptographic algorithm in their software supply chain**, which is the exact capability a CBOM provides.

Key compliance frameworks driving this: - **NSA CNSA 2.0** (Commercial National Security Algorithm Suite) mandates quantum-resistant algorithms for national security systems by 2030-2035 - **NIST SP 800-131A Rev 2** deprecates specific algorithm/key-size combinations - **Executive Order 14028** and subsequent OMB memoranda require federal agencies to inventory cryptographic assets

#### 1.2 Export Control (ECCN Classification)

Cryptographic software is subject to export controls under the **Wassenaar Arrangement** and national regulations (US EAR Category 5 Part 2, EU Dual-Use Regulation). Organizations that distribute software internationally must:

- Identify which cryptographic algorithms their software implements
- Determine the applicable **Export Control Classification Number (ECCN)**
- File license exceptions (e.g., EAR Section 740.17 for mass-market encryption)

Today, this is done manually through audits. A CBOM attached to the SBOM automates this inventory, reducing compliance cost and risk.

#### 1.3 Why Dependency-Track Is the Right Home

CBOMs are not a separate concept from SBOMs. They are **CycloneDX BOMs** with cryptographic-asset components and `cryptoProperties`. CycloneDX 1.6 already defines the schema. This means:

- The same upload flow handles both SBOMs and CBOMs
- The same project/portfolio model applies
- The same policy engine can enforce crypto-specific rules
- Users already comfortable with DT's vulnerability triage can apply the same workflow to crypto governance

For the DT community, CBOM support would: - **Expand the user base** to include cryptography compliance teams (post-quantum, export control, FIPS validation) - **Increase DT's relevance** as organizations adopt CycloneDX 1.6+ with crypto extensions - **Differentiate DT** from commercial alternatives that lack native CBOM handling

### 2. What We Propose

We propose adding first-class CBOM support to Dependency-Track. At its core, this introduces six capabilities:

1. **Crypto Inventory** — Ingest, store, and display every cryptographic asset from CycloneDX 1.6 CBOMs: algorithms, protocols, certificates, and key material metadata.
2. **Crypto Policy Enforcement** — Define rules to flag weak, deprecated, or non-compliant cryptography, using the same policy engine that handles vulnerabilities and licenses.
3. **Crypto Asset Triage** — Mark each detected crypto asset as in-use, not-in-use, or false-positive, following the same workflow used for vulnerability triage.
4. **Crypto Metrics** — Track per-project and portfolio-wide counters: total assets, quantum-vulnerable algorithms, expiring certificates, policy violations.
5. **CBOM Export** — Export the crypto inventory as valid CycloneDX 1.6 JSON for compliance audits and downstream tooling.
6. **Stable Cross-Project Identity** — Recognize the same algorithm across projects via deterministic identifiers, enabling portfolio-level crypto visibility.

This section provides a detailed technical design based on a proof-of-concept implementation built against v4.13.6. The PoC covers ingestion, storage, API, UI, policy engine, metrics, and export. Below we describe what the full, production-quality implementation should include, incorporating lessons from the PoC and additional requirements not covered in it.

### 3. Detailed Technical Design

#### 3.1 Data Model

The CBOM data model introduces seven new tables and modifies one existing table (**POLICYVIOLATION**). The design follows the same patterns used for components: a root entity linked to the project, type-specific sub-entities for different property sets, a triage/analysis entity, and a metrics table.

#### 3.2 Stable Identity for Crypto Assets

**3.2.1 The Problem: Crypto Assets Lack a Natural Identifier** For software components, Dependency-Track uses **Package URL (PURL)**, **CPE**, or **group/name/version** coordinates as stable identity keys. These identifiers are deterministic, `pkg:npm/lodash@4.17.21` is the same string regardless of which tool generated the BOM or when. DT uses this identity to match incoming components against existing ones during BOM re-uploads, preserving analysis decisions, policy violations, and audit trails across uploads.

Cryptographic assets have no equivalent to PURL. This creates three problems:

- **Re-upload data loss:** Without a stable identity, the only safe ingestion strategy is clean-slate (delete all, re-create all). This destroys all triage decisions (`IN_USE`, `NOT_IN_USE`, etc.) on every BOM re-upload.
- **Cross-project correlation:** Two projects both using AES-256-GCM have no shared key to correlate on. Portfolio-level questions like “which projects use quantum-vulnerable algorithms?” require fuzzy string matching instead of exact lookups.
- **Audit trail fragility:** If a crypto asset’s database record is deleted and recreated on each upload, its history (analysis states, comments, timestamps) is lost.

**3.2.2 CycloneDX Cryptography Definitions Registry** The CycloneDX project maintains a `cryptography-defs.json` registry that defines standardized naming patterns for algorithm families:

```
Family: AES      -> Pattern: AES[-(128|192|256)][-(ECB|CBC|CTR|CFB|OFB|GCM|CCM|...)]
Family: RSA-OAEP -> Pattern: RSA-OAEP[-{hashAlgorithm}][-{maskGenAlgorithm}][-{keyLength}]
Family: EdDSA    -> Pattern: Ed(25519|448)[- (ph|ctx)]
```

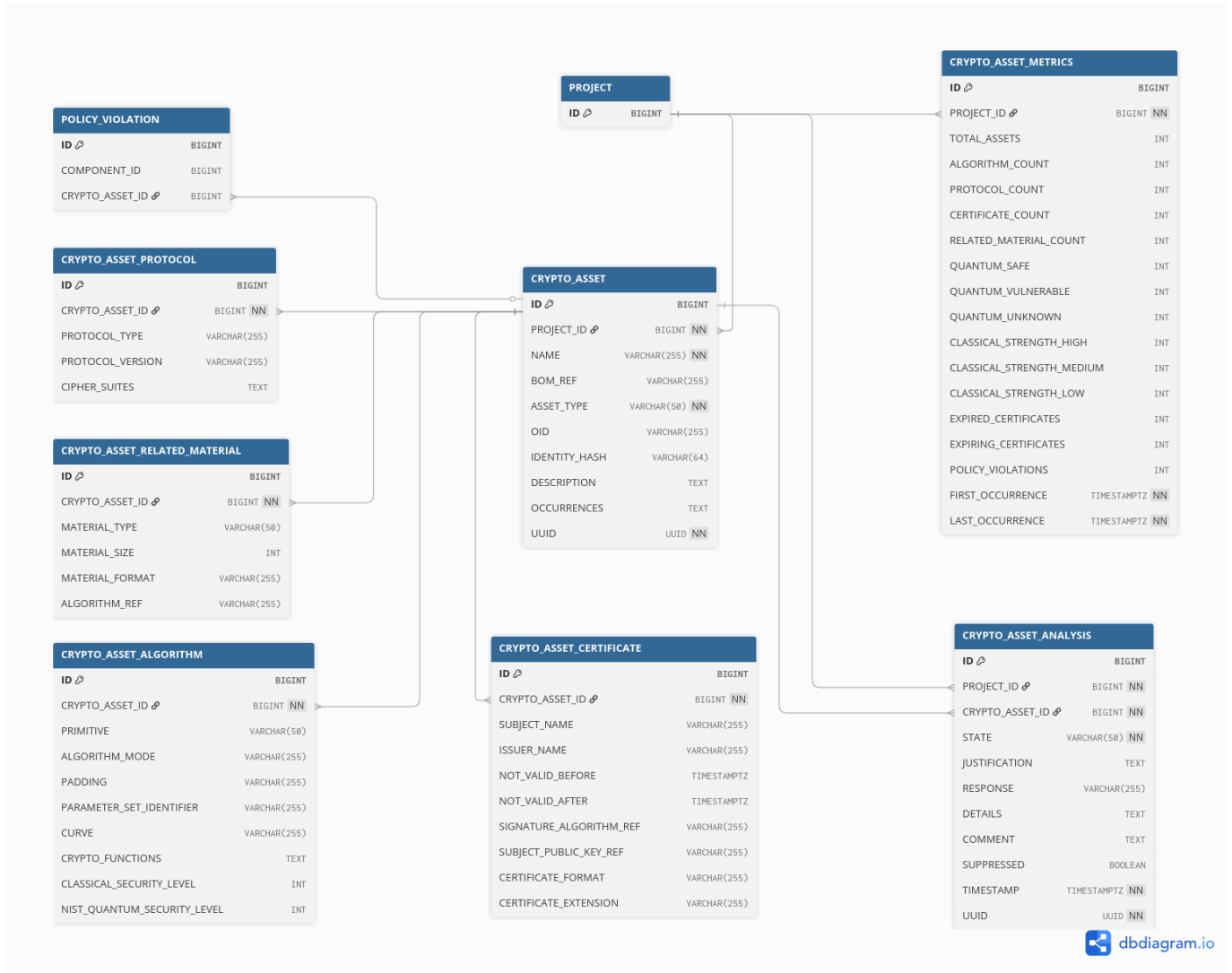


Figure 1: CBOM Entity Relationship Diagram

These patterns provide **canonical algorithm names** (e.g., AES-256-GCM, RSA-OAEP-SHA256-2048). However, they only cover algorithms, not certificates, protocols, or related material. They also do not define an identifier scheme; they define how to *name* an algorithm consistently.

**3.2.3 Proposed Approach: CryptoAssetIdentity with Content-Addressable Hash** Following DT's existing pattern for components (ComponentIdentity), we propose a two-part solution:

**1. A CryptoAssetIdentity class:** a transient object that computes identity from canonical properties, used during BOM processing to match incoming assets against existing database records. This mirrors how ComponentIdentity uses PURL/CPE/coordinates to match components.

The identity tuple depends on the asset type:

| Asset Type       | Identity Fields                                                      |
|------------------|----------------------------------------------------------------------|
| ALGORITHM        | name + assetType + primitive + mode + padding + parameterSetId + oid |
| CERTIFICATE      | subjectName + issuerName + notValidAfter + certificateFormat         |
| PROTOCOL         | protocolType + protocolVersion                                       |
| RELATED_MATERIAL | materialType + algorithmRef + materialFormat + materialSize          |

**2. A stored IDENTITY\_HASH column** on the CRYPTOASSET table – a SHA-256 hash (truncated to 16 hex chars) computed from the canonical identity tuple above. All values are lowercased and sorted before hashing.

Input: "algorithm:aes-256-gcm:block\_cipher:gcm:::"

SHA256: a1b2c3d4e5f6...

IDENTITY\_HASH: "a1b2c3d4e5f67890"

**Why both?** The transient CryptoAssetIdentity is used during BOM processing for identity-based matching (like ComponentIdentity). The stored IDENTITY\_HASH enables efficient portfolio-level queries: SELECT \* FROM CRYPTOASSET WHERE IDENTITY\_HASH = ? across all projects without needing to join and compare multiple fields.

**How BOM re-upload works with this approach:**

1. Parse incoming crypto assets from the new BOM
2. Compute CryptoAssetIdentity for each incoming asset
3. Load existing crypto assets for the project, indexed by identity
4. **Match found** -> Update existing record in-place (preserves UUID, analysis, audit trail)
5. **No match** -> Create new record
6. **Existing record not in new BOM** -> Delete it

This is the same algorithm DT uses for components at BomUploadProcessingTask.processComponents().

### 3.3 Ingestion Pipeline

**3.3.1 Parser** Extend ModelConverter.java with a convertCryptoAssetsFromJson(byte[] bomBytes, Project project) method. This method uses org.json to parse raw BOM JSON rather than the CycloneDX Java library, because the library does not yet fully support cryptographic-asset component types or cryptoProperties extraction. When the CDX library adds full support, the parser should be migrated.

The existing `convertComponents()` method filters out `cryptographic-asset` type components (using the CycloneDX library's `Component.Type.CRYPTOGRAPHIC_ASSET` enum), preventing them from being processed as regular software components. This ensures clean separation between the two pipelines.

#### Parsing steps:

1. Parse BOM bytes as JSON, iterate over `components[]`
2. Filter entries where `type == "cryptographic-asset"`
3. Map `cryptoProperties.assetType` to `CryptoAssetType` enum
4. Based on asset type, extract the corresponding sub-entity properties:
  - **Algorithm:** primitive, mode, padding, parameterSetIdentifier, curve, cryptoFunctions (stored as JSON), classicalSecurityLevel, nistQuantumSecurityLevel
  - **Certificate:** subjectName, issuerName, notValidBefore/After (ISO-8601), signatureAlgorithmRef, subjectPublicKeyRef, certificateFormat, certificateExtension
  - **Protocol:** type, version, cipherSuites (stored as JSON array)
  - **Related Material:** type, size, format, algorithmRef. **Security:** raw value field is stripped and never persisted (see Section 3.12)
5. Compute `CryptoAssetIdentity` and `identityHash` for each asset (see Section 3.2)
6. Resolve cross-references: `signatureAlgorithmRef`, `subjectPublicKeyRef`, and `algorithmRef` are stored as bom-ref strings pointing to other crypto assets in the same BOM

**Error handling:** Individual asset parsing failures are logged at WARN level with the asset name and error detail, but do not abort BOM processing. Unrecognized enum values (e.g., an unknown `primitive`) are logged and the field is set to null rather than defaulting to a potentially misleading value.

**3.3.2 BOM Upload Integration** Add `processCryptoAssets(ctx)` to `BomUploadProcessingTask`, called after main component processing:

- **Separate transaction:** Runs in its own `QueryManager` and transaction to isolate crypto processing failures from SBOM ingestion. If crypto parsing fails, the SBOM portion of the upload is not rolled back.
- **Identity-based matching:** Follows the same algorithm described in Section 3.2.3, mirroring how `processComponents()` handles software components:
  1. Parse incoming crypto assets and compute `CryptoAssetIdentity` for each
  2. Load existing crypto assets for the project, indexed by identity
  3. For each incoming asset: if a match exists, update in-place (preserving UUID, analysis records, and audit trail); if no match, create new
  4. Delete existing assets not present in the new BOM
- **Early return:** When the BOM contains no `cryptographic-asset` components, returns immediately with a debug log. Zero impact on non-CBOM uploads.
- **Event dispatch:** After successful persistence, dispatches `CryptoMetricsUpdateEvent` to trigger asynchronous metrics computation.

**3.3.3 Crypto-Only BOM Edge Case** When a BOM contains crypto assets but no regular software components, the normal processing chain (vulnerability analysis completion -> policy evaluation trigger) does not fire. The ingestion pipeline handles this by:

1. Marking vulnerability analysis workflow status as `NOT_APPLICABLE`
2. Dispatching `ProjectPolicyEvaluationEvent` directly, so crypto policy conditions are still evaluated

### 3. Dispatching ProjectMetricsUpdateEvent as usual

This ensures crypto-only CBOMs receive full policy evaluation without requiring a dummy component.

## 3.4 REST API

### 3.4.1 Crypto Asset Endpoints All require VIEW\_CRYPT0\_ASSETS unless noted.

| Endpoint                                     | Description                                     |
|----------------------------------------------|-------------------------------------------------|
| GET /v1/crypto-asset/project/{uuid}          | List assets for project (paginated, filterable) |
| GET /v1/crypto-asset/{uuid}                  | Get single crypto asset                         |
| GET /v1/crypto-asset/{uuid}/algorithm        | Algorithm detail                                |
| GET /v1/crypto-asset/{uuid}/certificate      | Certificate detail                              |
| GET /v1/crypto-asset/{uuid}/protocol         | Protocol detail                                 |
| GET /v1/crypto-asset/{uuid}/related-material | Related material detail                         |
| DELETE /v1/crypto-asset/{uuid}               | Delete asset (requires PORTFOLIO_MANAGEMENT)    |

### 3.4.2 Analysis Endpoints

| Endpoint                               | Description                              |
|----------------------------------------|------------------------------------------|
| GET /v1/crypto-analysis/project/{uuid} | List analyses (VIEW_CRYPT0_ASSETS)       |
| PUT /v1/crypto-analysis                | Create/update analysis (CRYPTO_ANALYSIS) |
| DELETE /v1/crypto-analysis/{uuid}      | Delete analysis (CRYPTO_ANALYSIS)        |

### 3.4.3 Metrics Endpoints

| Endpoint                              | Description                  |
|---------------------------------------|------------------------------|
| GET /v1/metrics/crypto/project/{uuid} | Project crypto metrics       |
| GET /v1/metrics/crypto/portfolio      | Portfolio aggregated metrics |

### 3.4.4 Export Endpoints

| Endpoint                          | Description                              |
|-----------------------------------|------------------------------------------|
| GET /v1/bom/cbom/project/{uuid}   | Export project CBOM (CycloneDX 1.6 JSON) |
| GET /v1/bom/cbom/component/{uuid} | Export component scoped CBOM             |
| POST /v1/bom/cbom/portfolio       | Export portfolio CBOM                    |

All export endpoints require VIEW\_CRYPT0\_ASSETS. Query parameters: includeDependencies, includeAnalysis, includePolicyViolations, minify.

## 3.5 Policy Engine

### 3.5.1 New Policy Condition Subjects 9 new subjects added to PolicyCondition.Subject:

| Subject                        | Evaluates                       | Operators           |
|--------------------------------|---------------------------------|---------------------|
| CRYPTO_ALGORITHM_NAME          | Asset name                      | IS, IS_NOT, MATCHES |
| CRYPTO_ALGORITHM_PRIMITIVE     | Primitive enum                  | IS, IS_NOT, MATCHES |
| CRYPTO_ALGORITHM_PARAMETER_SET | Parameter set ID                | IS, IS_NOT, MATCHES |
| CRYPTO_QUANTUM_SECURITY        | NIST quantum security level     | All numeric         |
| CRYPTO_CLASSICAL_STRENGTH      | Classical security level (bits) | All numeric         |
| CRYPTO_PROTOCOL_VERSION        | Combined “type version” string  | IS, IS_NOT, MATCHES |
| CRYPTO_CERTIFICATE_EXPIRY      | Days until expiry               | All numeric         |
| CRYPTO_CERTIFICATE_ALGORITHM   | Signature algorithm ref         | IS, IS_NOT, MATCHES |
| CRYPTO_MATERIAL_TYPE           | Material type enum              | IS, IS_NOT, MATCHES |

Each subject is handled by a dedicated evaluator class (e.g., `CryptoAlgorithmPolicyEvaluator` for the algorithm subjects, `CryptoQuantumPolicyEvaluator` for quantum/classical strength).

**3.5.2 Pre-Configured Policy Templates** The following policies should be shipped as seed data, ready for users to enable:

| Policy                  | Condition                                               | Use Case                       |
|-------------------------|---------------------------------------------------------|--------------------------------|
| Quantum Vulnerable      | CRYPTO_QUANTUM_SECURITY < 1                             | Post-quantum migration         |
| Weak Classical Strength | CRYPTO_CLASSICAL_STRENGTH < 128                         | Security hygiene               |
| Legacy Algorithms       | CRYPTO_ALGORITHM_NAME matches MD5, SHA1, DES, 3DES, RC4 | Deprecated algorithm detection |
| Expiring Certs (30d)    | CRYPTO_CERTIFICATE_EXPIRY < 30                          | Certificate lifecycle          |
| Legacy TLS              | CRYPTO_PROTOCOL_VERSION matches TLS 1.0/1.1             | TLS compliance                 |
| Non-CNSA 2.0            | CRYPTO_ALGORITHM_NAME matches non-CNSA regex            | CNSA 2.0 compliance            |

**3.5.3 Violation Types** We recommend adding `SECURITY_CRYPT0` and `OPERATIONAL_CRYPT0` to `PolicyViolation.Type` to distinguish crypto policy violations from vulnerability-related security violations. This enables filtered views and reporting that separate crypto governance from vulnerability management.

## 3.6 UI

### 3.6.1 Navigation

- New top-level sidebar entry: **Crypto Assets** (lock icon), gated by `VIEW_CRYPT0_ASSETS` permission
- Route: `/cryptoAssets`

### 3.6.2 Views

| View                          | Description                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Crypto Assets Overview</b> | Top-level list with type filter (Algorithm/Protocol/Certificate/Related Material), color-coded badges for type and quantum status, server-side pagination |

| View                                 | Description                                                                                                          |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Project Crypto Tab</b>            | New tab on project page showing project-scoped crypto assets with CBOM export button                                 |
| <b>Algorithm Sub-View</b>            | Filtered view showing only algorithms with primitive, mode, key size, quantum level columns                          |
| <b>Protocol Sub-View</b>             | Filtered view showing protocols with type, version, cipher suite summary                                             |
| <b>Crypto Audit View</b>             | Analysis/triage decisions for crypto assets (see Section 3.7)                                                        |
| <b>Crypto Policy Violations View</b> | Crypto-specific policy violations with triage (see Section 3.8)                                                      |
| <b>Dashboard Widgets</b>             | Crypto overview cards on project dashboard: totals by type, quantum readiness pie chart, expiring certificates count |

### 3.7 Crypto Asset Triage

**3.7.1 Motivation** When a CBOM is ingested, all detected cryptographic components appear in the inventory. However, not all of them are necessarily *in active use* in the software. For example:

- A library may ship with support for multiple algorithms, but the application only configures one
- A dependency may include deprecated cipher suites that are compiled-in but never negotiated
- A certificate bundled in a container image may belong to a test environment

Users need the ability to **triage** each detected crypto component to indicate whether it is actively used, irrelevant, or still under investigation. This is directly analogous to how DT handles vulnerability triage today.

**3.7.2 Proposed Analysis States** We propose the following states for crypto asset analysis, inspired by but distinct from DT's existing vulnerability analysis states:

| State          | Meaning                 | When to Use                                                        |
|----------------|-------------------------|--------------------------------------------------------------------|
| NOT_SET        | No decision made yet    | Default for all newly ingested assets                              |
| IN_TRIAGE      | Under investigation     | Analyst is reviewing whether the asset is in use                   |
| IN_USE         | Confirmed actively used | The crypto component is confirmed to be exercised at runtime       |
| NOT_IN_USE     | Confirmed not used      | Present in code/dependencies but not reachable or configured       |
| FALSE_POSITIVE | Detection was incorrect | CBOM generator incorrectly identified this as a crypto asset       |
| RESOLVED       | Issue addressed         | Deprecated algorithm has been replaced, expired cert renewed, etc. |

**Justification field** (free text or enum) to record *why* the analyst made that decision. Examples: - “Only AES-256-GCM is configured in production; AES-128-CBC is unused legacy code” - “This certificate is for the test environment only”

**3.7.3 Architecture** This should follow the existing **Analysis / AnalysisComment** pattern used for vulnerability triage:



| DT Vulnerability Triage           | Crypto Asset Triage (Proposed)                      |
|-----------------------------------|-----------------------------------------------------|
| Analysis model                    | CryptoAssetAnalysis model                           |
| AnalysisState enum                | CryptoAssetAnalysisState enum (states above)        |
| AnalysisComment model             | CryptoAssetAnalysisComment model                    |
| AnalysisJustification enum        | CryptoAssetJustification enum (optional, see below) |
| AnalysisResource REST             | CryptoAnalysisResource REST                         |
| VULNERABILITY_ANALYSIS permission | CRYPTO_ANALYSIS permission                          |

### Key implementation details:

- **Composite unique constraint** on (project, cryptoAsset) ensures one analysis record per asset per project
- **Immutable audit trail** via CryptoAssetAnalysisComment: every state change auto-generates a comment with timestamp and commenter (e.g., "IN\_TRIAGE -> IN\_USE")
- **Suppression flag** (suppressed boolean) allows hiding triaged assets from active counts without deleting the record
- **REST API** follows the PUT /v1/crypto-analysis upsert pattern, where null fields in the request preserve existing values

**3.7.4 Optional: Structured Justification Enum** For organizations that need structured triage reasons (e.g., for auditors), an optional CryptoAssetJustification enum:

| Value                  | Description                                                          |
|------------------------|----------------------------------------------------------------------|
| CODE_NOT_REACHABLE     | Algorithm in dependency but execution path not reachable             |
| CONFIGURATION_DISABLED | Algorithm compiled in but disabled via configuration                 |
| TEST_ENVIRONMENT_ONLY  | Certificate/key used only in non-production environments             |
| DEPENDENCY_MANAGED     | Crypto usage managed by upstream dependency, not directly controlled |
| MIGRATION_PLANNED      | Deprecated algorithm acknowledged, migration scheduled               |
| ACCEPTED_RISK          | Known weak algorithm accepted per risk assessment                    |

## 3.8 Policy Violation Triage

**3.8.1 Motivation** Crypto policies (Section 3.5) will inevitably generate false positives. For example:

- A “Quantum-Vulnerable Algorithms” policy flags `RSA-2048` in a non-production test tool
- A “Legacy TLS” policy flags `TLS 1.1` support in a library that also supports `TLS 1.3` and only negotiates the latter
- A “Weak Classical Strength” policy flags an algorithm that is only used for non-security-critical checksumming

Users must be able to **triage these violations** without disabling the policy entirely. This is not a new concept. DT already supports it for license and security policy violations via the `ViolationAnalysis` model.

**3.8.2 Proposed Approach: Reuse Existing ViolationAnalysis** Crypto policy violations should use the **existing ViolationAnalysis / ViolationAnalysisState** mechanism without modification. This is the correct approach because:

1. **Crypto violations are PolicyViolation objects**, created by the policy engine just like license or security violations

## 2. The existing triage states apply directly:

| ViolationAnalysisState | Meaning for Crypto Violations                                                |
|------------------------|------------------------------------------------------------------------------|
| NOT_SET                | No decision on this violation yet                                            |
| APPROVED               | Violation acknowledged and accepted (e.g., risk accepted, exception granted) |
| REJECTED               | Violation confirmed as real; must be remediated                              |

3. **The existing UI already works.** The policy violations view shows all violation types with their analysis states. Adding `SECURITY_CRYPT0` / `OPERATIONAL_CRYPT0` violation types automatically surfaces crypto violations in this view with full triage support.
4. **Audit trail is automatic.** `ViolationAnalysisComment` records all state transitions with timestamp and commenter.

### 3.8.3 What Needs to Be Done

- No new models needed. The existing `ViolationAnalysis`, `ViolationAnalysisState`, and `ViolationAnalysisComment` classes handle everything.
- The `ViolationAnalysisResource` REST API (`PUT /v1/violation/analysis`) already supports triaging any `PolicyViolation` regardless of type.
- The only requirement is that crypto policy evaluators generate standard `PolicyViolation` objects (which they do), so the violation analysis machinery applies automatically.
- The UI should ensure crypto violations are clearly identifiable (via the `SECURITY_CRYPT0` type badge) so analysts can filter and triage them efficiently.

## 3.9 Permissions

Two new permissions:

| Permission                      | Description                                               |
|---------------------------------|-----------------------------------------------------------|
| <code>VIEW_CRYPT0_ASSETS</code> | View cryptographic asset inventory, analysis, and metrics |
| <code>CRYPT0_ANALYSIS</code>    | Create/update/delete analysis decisions on crypto assets  |

Policy management and violation analysis reuse the existing `POLICY_MANAGEMENT` and `POLICY_VIOLATION_ANALYSIS` permissions.

## 3.10 Metrics

### 3.10.1 Per-Project Crypto Metrics (14 counters)

| Counter                           | Description                                         |
|-----------------------------------|-----------------------------------------------------|
| <code>totalAssets</code>          | Total crypto assets                                 |
| <code>algorithmCount</code>       | Assets of type <code>ALGORITHM</code>               |
| <code>protocolCount</code>        | Assets of type <code>PROTOCOL</code>                |
| <code>certificateCount</code>     | Assets of type <code>CERTIFICATE</code>             |
| <code>relatedMaterialCount</code> | Assets of type <code>RELATED_CRYPT0_MATERIAL</code> |

| Counter                 | Description                                                  |
|-------------------------|--------------------------------------------------------------|
| quantumSafe             | Algorithms with NIST quantum security level $\geq$ threshold |
| quantumVulnerable       | Algorithms with quantum level $<$ threshold                  |
| quantumUnknown          | Algorithms with null/unknown quantum level                   |
| classicalStrengthHigh   | Algorithms with classical level $\geq$ 192 bits              |
| classicalStrengthMedium | Algorithms with $128 \leq$ classical level $<$ 192           |
| classicalStrengthLow    | Algorithms with classical level $<$ 128 bits                 |
| expiredCertificates     | Certificates past <code>notValidAfter</code>                 |
| expiringCertificates    | Certificates expiring within 30 days                         |
| policyViolations        | Count of crypto-related policy violations                    |

**3.10.2 Metrics Task** `CryptoMetricsUpdateTask` subscribes to `CryptoMetricsUpdateEvent`, computes all counters, and persists with deduplication (only creates a new row when values change; otherwise updates `lastOccurrence`).

### 3.10.3 Recommended Extensions

- Extend `ProjectMetrics` with summary crypto fields (`cryptoAssets`, `cryptoQuantumVulnerable`, `cryptoPolicyViolations`)
- Add portfolio-level crypto metrics aggregation
- Add triage state distribution to metrics (count of `IN_USE`, `NOT_IN_USE`, `IN_TRIAGE`, etc.)

## 3.11 CBOM Export

`CbomExportResource` generates CycloneDX 1.6 JSON with:

- `bomFormat`: "CycloneDX", `specVersion`: "1.6"
- `serialNumber` as UUID URN (per CycloneDX spec requirement)
- `metadata.component` with project details
- `components[]` array where each entry has:
  - `type`: "cryptographic-asset"
  - `bom-ref` (original or deterministic, per Section 3.2)
  - `name`, `description`
  - `cryptoProperties` with all type-specific sub-properties
  - Optionally: analysis annotations, policy violation annotations

## 3.12 Security Considerations

| Concern             | Mitigation                                                                                                                                |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Private keys in BOM | Parser strips <code>relatedCryptoMaterialProperties.value</code> if present; logs warning; persists only metadata (type, size, format)    |
| RBAC enforcement    | All endpoints gated by <code>VIEW_CRYPTO_ASSETS</code> or <code>CRYPTO_ANALYSIS</code> ; write operations require appropriate permissions |
| XSS in crypto names | Frontend uses <code>xssFilters.inHTMLData()</code> on all dynamic values                                                                  |

## 4. Implementation Phases

We recommend a phased approach that delivers incremental value:

### Phase 1: Data Foundation (Database + Models)

- JDO model classes for all 7 entities + 5 enums
- CryptoAssetQueryManager with full CRUD
- persistence.xml registration
- CryptoAssetIdentity class and identityHash computation utility (see Section 3.2)
- **Success criteria:**  $\geq 80\%$  model test coverage; schema auto-creates correctly

### Phase 2: Ingestion Pipeline

- ModelConverter parser extension for cryptographic-asset components
- BomUploadProcessingTask integration
- Sensitive material stripping
- **Success criteria:** Sample CBOMs ingest correctly; non-CBOM BOMs unaffected

### Phase 3: REST API + Auth

- All CRUD, analysis, metrics, and export endpoints
- Permission enforcement
- OpenAPI annotations
- **Success criteria:** Full API coverage verified by integration tests

### Phase 4: Policy Engine

- 5 evaluators, 9 subjects
- Policy seed data / templates
- SECURITY\_CRYPT0 violation type
- **Success criteria:** Pre-configured policies detect weak algorithms, expiring certs with  $< 1\%$  false negatives on curated test set

### Phase 5: UI

- Top-level Crypto Assets view with filtering
- Project-level crypto tab
- Crypto asset triage panel (analysis states, comments, suppression)
- Policy violation triage (reuses existing violation analysis UI)
- Dashboard widgets
- **Success criteria:** UX review sign-off; all views gated by permissions

### Phase 6: Metrics + Export Extensions

- Portfolio-level metrics
- Dashboard metrics endpoint
- Component-scope and portfolio-scope export
- Export query parameters (dependencies, analysis, violations, minify)

## 5. Effort Estimate

| Phase                              | Traditional (1 developer) | With AI-assisted development |
|------------------------------------|---------------------------|------------------------------|
| Phase 1: Data Foundation           | 1-2 weeks                 | 2-3 days                     |
| Phase 2: Ingestion                 | 1-2 weeks                 | 2-3 days                     |
| Phase 3: REST API                  | 1 week                    | 1-2 days                     |
| Phase 4: Policy Engine             | 1-2 weeks                 | 2-3 days                     |
| Phase 5: UI                        | 2-3 weeks                 | 1-2 weeks                    |
| Phase 6: Metrics + Export          | 1-2 weeks                 | 2-3 days                     |
| <b>Testing &amp; Documentation</b> | 2-3 weeks                 | 1-2 weeks                    |
| <b>Total</b>                       | <b>10-16 weeks</b>        | <b>4-6 weeks</b>             |

## 6. Proof of Concept

We have built a working PoC against Dependency-Track v4.13.6 that implements Phases 1-5 (excluding some sub-views and dashboard widgets). The PoC includes:

- **24 new Java files** (models, enums, persistence, policy evaluators, REST resources, metrics task)
- **7 modified Java files** (QueryManager, PolicyCondition, PolicyEngine, Permissions, ModelConverter, BomUploadProcessingTask, persistence.xml)
- **2 new Vue components + 7 modified frontend files**
- Full patches are available for review

The PoC demonstrates that CBOM support integrates cleanly into DT's existing architecture without disrupting SBOM workflows.

## 7. References

- CycloneDX Specification - Cryptographic Assets
- CycloneDX Cryptography Definitions Registry
- NIST Post-Quantum Cryptography Standards (FIPS 203, 204, 205)
- NSA CNSA 2.0 Algorithm Suite
- EAR Category 5 Part 2 - Encryption Items
- IBM CBOMkit